SereneDB challenge · SchubKI National AI Hackathon 2026

Making Sense of Logs

Ask your logs a question in plain words. The engineer, the team lead and the manager each get the answer they need, and every number is checked against the query that produced it.

Amirali & Konark

5 M

log lines, BGL + OpenStack

44

MCP tools for Claude

62 / 62

numbers traced to a query

THE PROBLEM

Logs hold the answers, but nobody can read them

Three people ask about the same logs. Today each of them works by hand.

Engineer

“What failed, and what happened just before?”

Greps across millions of lines while the system is down, then pieces the timeline together by hand.

Team lead

“What keeps coming back, and whose is it?”

Counts in spreadsheets. One noisy storm looks like a thousand incidents.

Manager

“What do we fix first?”

Priorities set by whoever is loudest, not by measured frequency, spread and trend.

Why an AI chatbot alone does not fix it: 4.7 M lines do not fit in a prompt, and a model that guesses numbers is worse than no answer.

THE PROBLEM

Raw logs mislead, even when you can read them

Measured on the brief's BGL supercomputer logs.

855,501

FATAL lines

but only 36,714 bursts of recurring problems over all 4.7 M lines. Lines are not incidents.

1 / 10

real alerts

in the top 10 when you rank by log level. The loudest messages are routine register dumps.

69.6 %

claims correct

when plain Claude Code greps the raw files itself (pilot, 3 questions). With our server: 94.3 %.

THE SOLUTION

The database counts, the AI explains

A question goes in. Evidence comes from SereneDB. The answer is written for the role and checked.

1 · Ask

In plain words, in Claude Code (or any MCP client).

2 · Pick tools

Claude chooses from 44 safe, fixed queries. It never writes SQL.

3 · SereneDB

Searches and counts every line: full text, timelines, trends, rankings.

4 · Jev

Already knows which kinds of messages are real faults.

5 · Answer

Facts, a labelled hypothesis, unknowns and a next step. Every number is linked to its query.

Setup: `docker compose up -d` · `claude` · `“Load the base corpus”` → 5 M lines ready in 72 s.

DEMO

One problem, three different answers

BGL supercomputer, November 2005: the top-ranked problem, seen by each role.

Engineer

What failed, before and after?

- New fault “Link has been severed”: 12,996 lines on 1,024 nodes
- First burst hit 504 nodes in 4 minutes; 21 minutes earlier, 64 job launches failed
- Every line points to one host: 172.16.96.116

Next check: that host's log, 05:30–05:40

Team lead

What recurs, and whose is it?

- 451 problem bursts: node-platform 283, hpc-workloads 106, cluster-control 62
- The new fault: 31 bursts; its owner is disputed between two teams
- Weekly problem bursts 69.6 → 119.3

Next step: both teams triage it together

Manager

What do we fix first?

- Computed rank 1: the new fault (score 97.8)
- Ranks 2 and 3: two kernel faults (93.6, 90.5)
- hpc-workloads: 9 → 106 bursts vs the month before

Decision: name one owner now. Cost and downtime are not in the logs.

WHY IT IS MORE ACCURATE

Numbers come from the database, not the model

Every number traced

62 of 62 numbers in the worked investigation are found in a tool result. A Verify button re-runs each query: 17 of 17 give the same result.

Bursts, not lines

Tools count recurrences and affected nodes, so a storm of 12,996 lines reads as 31 bursts, not 12,996 incidents.

Real faults first

Jev's judgment per kind of message lifts real alerts in the top 10 from 1 to 5.

Facts vs guesses

Facts, a labelled hypothesis, what the logs do not show, and a next step, every time.

Measured on 50 golden questions

13 – 2

wins vs the brief's reference agent on BGL + OpenStack (blind Opus judge)

53 vs 69

wrong claims over 50 answers, checked against frozen answer keys

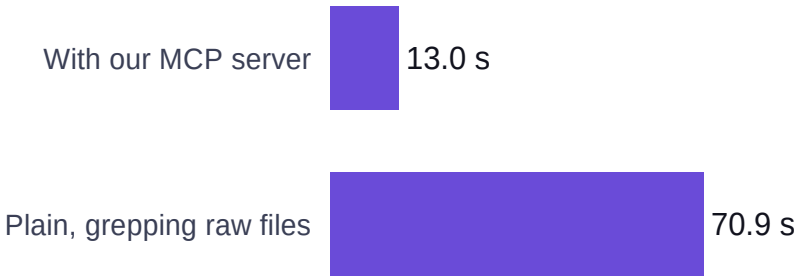
Full results: docs/dashboard.html

WHY IT IS FASTER

The database does the reading

Claude reads a few results instead of grepping millions of lines.

Claude Code, same model, same questions (pilot, 3 questions)



Mean seconds per answer.
Claims correct: 94.3 % vs 69.6 %.

1.8 s

of database time for all 20 tool calls of the worked investigation

7 ms

to count 121,498 full-text matches over 4.7 M lines (inverted index, no scan)

~90 %

of an answer's time is the AI writing; SereneDB is under 1 s per answer

HOW IT FITS THE COMPANY

It plugs into routines that already exist

On-call incident

The engineer asks during the outage and gets the failure, its neighbours and a next check in about 13–44 s, instead of a manual search.

Weekly team review

The lead opens recurring problems by team and trend for a stated period, ready for the meeting.

Monthly planning

The manager gets a computed ranking with owners and a decision. It never invents cost or downtime.

Tools they already use Claude Code, Claude Desktop or any MCP client.

Their data stays home SereneDB runs locally in Docker; only tool results reach the AI.

Their own logs Any file; the format is detected from 76 known formats in seconds.

Their org chart An ownership table maps components to teams; access per team is enforced by the database.

CHEAPER THAN THE ALTERNATIVES

Judge each kind of message once, not every line

Jev is a small model that gives typed, calibrated judgments. It runs inside SereneDB with one SQL call.

Reading BGL once with an LLM

≈ 177 M

input tokens (709 MB ÷ ~4 bytes per token, an estimate), and again for every new question or new day of logs.

Jev on the 1,126 kinds of message

≈ 0.5 M tokens · 20 s · ≈ $0.02 once, stored, reused by every answer

Measured, not decorative
Real alerts in the priority top 10: 1 → 5 after Jev's judgment.

No model at question time
Tools read the stored judgments, so answering needs no extra classification calls.

Probabilities, not vibes
Each judgment and each fact's support is a calibrated probability, never self-reported confidence.

Pennies per answer
Claude Code + our server: about $0.07 per answer in the pilot.

PERFORMANCE

Built on SereneDB's search-analytics engine

Every feature below is used by a tool, and every number is measured on the base corpus.

Inverted index + BM25
full-text, phrase, fuzzy, top-k

7 ms
count over 4.7 M lines

Counts from the term dictionary
group-bys without scanning rows

3–39 ms
whole-corpus breakdowns

SQL-native load
read_csv + regex parse inside SereneDB

18 s
4.7 M BGL lines loaded

ASOF join · ROLLUP · PIVOT
precursors, subtotals, team × week

< 100 ms
per analytics tool

Remote Parquet over S3
row-group pruning, read in place

2.8 s
10 of 1,624 row groups

On-demand index
index a remote slice, search it

0.94 ms
vs 281 ms scan; pays off after 4 searches

LOADING

Loading is where the work happens, once

“Load the base corpus” or “Load ~/app.log”: every step runs inside SereneDB, so answering later is cheap.

1 · Stream in

COPY FROM STDIN from anywhere on disk, or the brief's Zenodo links

SereneDB COPY

2 · Detect

76 lnav formats scored on a 2,000-line sample; loaded only at ≥ 95 %

SQL regex scoring

3 · Parse

regex + strptime into typed columns; ids, IPs, paths extracted

read_csv · regexp

4 · Patterns

numbers / ids masked → 1,126 kinds of message; bursts by window LAG

window functions

5 · Jev judges

real fault or routine? owner area? one call per pattern

ai_system_one in SQL

6 · Index

one inverted index (BM25, phrase, fuzzy) + time-sorted storage

inverted index · zonemaps

72 s

clean clone → BGL + OpenStack ready (5 M lines)

18 s

4.7 M BGL lines parsed + indexed in SQL

20 s · $0.02

Jev judges all 1,126 BGL patterns

1.6 s

a new 21 k-line file, format auto-detected

One MCP server, four ways to use it

MCP is how Claude discovers and calls our functions. Ours offers all four kinds of MCP features.

44 tools

Search, count, timelines, patterns, priorities, ownership and setup. Claude picks them from their descriptions.

`Just ask in plain words`

5 prompts

Ready-made playbooks: one per role, a three-role investigation and whole-corpus counts.

`/mcp__logs__set_priorities`

6 resources

Read-only context: datasets, glossary, field guide, ownership and the session's full evidence.

`@logs:logs://ownership/bgl`

Completions

Prompt arguments complete with real values from the data: datasets, components, teams.

`Tab inside a prompt argument`

44 tools, grouped by the question they answer

Each description says when to use it, when not to, what it returns and which tool comes next.

Set up	`setup_status · load_logs · use_dataset · remove_dataset`	*Load my log file ~/app.log*
Find	`search_logs · get_context · find_patterns · suggest_terms · facet`	*What failed on R43 at 13:56?*
Count	`count_by · trend · facet · rollup · heatmap · rate_vs_baseline`	*Errors per team, week over week?*
Recurring	`top_patterns · pattern_history · whats_new · spread · periodicity`	*What keeps coming back?*
Explain	`burst_timeline · precursors · request_timeline · value_outliers`	*What happens just before it?*
Decide	`priority_table · reliability_trend · team_risk · owner_of`	*What first, and who owns it?*
Workspace	`create_focus_index · index_stats · explain_tool · session_report`	*Index this week for a deep dive*

Every result: the SQL that ran, filters, scope, the exact total, and a flag when rows were cut to 50. Reference: docs/TOOLS.md

How to use them in Claude Code

Prompts: type /

`/mcp__logs__investigate_failure`
engineer playbook

`/mcp__logs__review_recurring_problems`
team-lead playbook

`/mcp__logs__set_priorities`
manager playbook

`/mcp__logs__worked_investigation`
one topic, all three roles

`/mcp__logs__whole_corpus_counts`
counts over every dataset

Resources: type @

`logs://datasets`	what is loaded
`logs://glossary`	line, pattern, burst; the rules
`logs://field-guide/{dataset}`	what each field means
`logs://ownership/{dataset}`	component → team
`logs://session/latest`	every query so far
`logs://session/{id}/step/{n}`	one step in full

Completions Prompt arguments suggest real datasets, components, teams and levels as you type.

Connect, load, restrict, verify

Connect	In the repo, run claude (it reads .mcp.json). From anywhere: claude mcp add -s user logs -- uv run … python mcp_server/server.py
Load	load_logs('base') = BGL + OpenStack from the brief's Zenodo links in 72 s. load_logs('~/app.log') detects the format from 76 known ones and refuses below 95 % match.
Restrict	LOGS_DB_ROLE=team_<team>: the database shows a team only its own components. logs_manager is read-only.
Verify	session_report gives a local page: the answer, each number linked to its query, and a Verify button that re-runs each step read-only.

WOW

Beyond what the brief asked for

Bring your own logs

Any file, format detected, loaded in seconds (21 k lines in 1.6 s). 10 Loghub datasets, no code change.

A Verify button

Re-runs the query behind every number on a read-only connection.

Judged once, reused

Jev labels each kind of message inside the database; nothing re-reads raw lines.

1 billion remote rows

OpenTelemetry Parquet on S3, queried in place: no download.

Access per team

Enforced by database roles and views, not by the prompt.

Our own blind evaluation

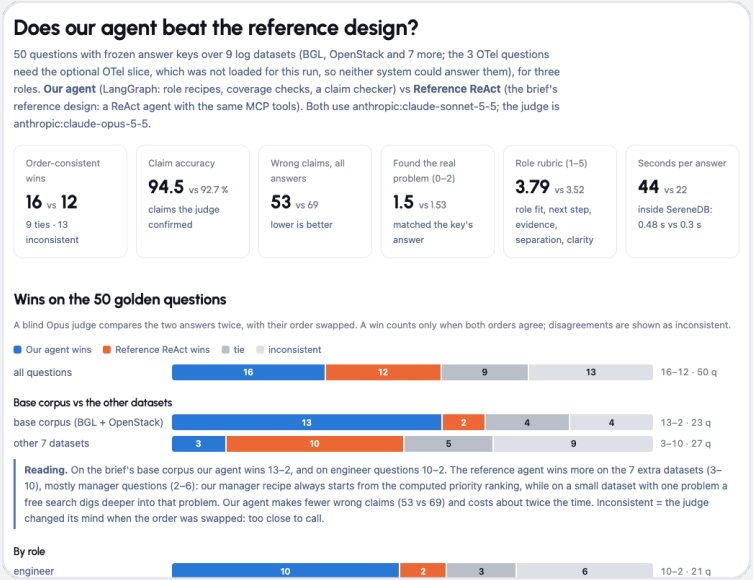
50 answer keys built from raw SQL and labels, an Opus judge, both answer orders.

WOW

Every answer comes with its proof



Session report each number links to the query behind it; Verify re-runs it.



Eval dashboard 50 questions, both answers, the judge's reasons, how to re-run.

SCALE

Where it goes next

Mittelstand

Machine and production logs, IT operations, managed service providers: the same tools on their files.

Other industries

Any system that writes logs: cloud, telecom, devices, security. SSH brute-force logs are already in our eval.

A product

A self-hosted log copilot: MCP server + SereneDB in one container, team access, alerts from saved searches.

10
log types loaded, 7 kinds of systems, no code change

11.2 M
HDFS lines loaded in 47 s

1 B
remote OTel rows, queried in place

Try it in two commands

```
docker compose up -d
claude          # in the repo, approve the logs server
> Load the base corpus
```

- docs/INVESTIGATION.md: the worked investigation and its evidence page
- docs/dashboard.html: the evaluation, every answer, how to re-run it
- docs/TOOLS.md: every tool, prompt and resource

github.com/aircode610/making-sense-of-logs-schubki-ai-hackathon